

La logique globale de la Mission C

Comprendre, d'un seul tenant, ce que construit **orchestrator-go** : un orchestrateur distribué qui garantit un cluster résilient — un seul écrivain, une réplication multi-site qui converge, une base de données haute disponibilité qui rebascule en moins de 5 secondes.

Un seul écrivain consensus etcd + fencing par lease — anti split-brain	Convergence AP delta-CRDT max-merge : doublons et coupures inoffensifs	HA < 5 s superviseur maison : détecter vite, laisser Patroni décider	Preuve continue tests à 3 niveaux + chaos + CI à seuils bloquants
---	---	--	--

Document généré le 24/08/2026 depuis le code du dépôt (sources : orchestrator-go/, proto/, scripts/, docker-compose.yml). Tous les schémas sont vectoriels et régénérables :

```
.venv/bin/python docs/generate_mission_c_logique_pdf.py
```

Sommaire

#	Section	Ce que vous y apprendrez
1	Le problème que résout Mission C	les quatre exigences et leur traduction technique
2	Vue d'ensemble : ce que fait le système	architecture physique, composants, plan mémoire etcd
3	Le contrat gRPC : la frontière versionnée	proto partagé B ↔ C / A ↔ C, règle de compatibilité buf
4	Un seul écrivain : consensus et fencing	élection par lease, perte de lease = verrouillage
5	Le chemin d'écriture : gated et idempotent	la garde IsLeader() avant tout accès au journal
6	Réplication multi-site : le delta-CRDT	totaux cumulés, fusion max, propagateur périodique
7	Haute disponibilité < 5 s : le superviseur	détection rapide, garde anti-partition, mesures
8	Mesh WireGuard intra-site	registre public, config /32, secrets locaux
9	Sécurité transverse	TLS 1.3 + mTLS, zéro secret en clair, payload chiffré
10	Observabilité : la trace distribuée W3C	traceparent de nœud en nœud, logs corrélés
11	La preuve par les tests et la CI	pyramide de tests, chaos, seuils bloquants
12	Synthèse : six principes directeurs	la logique d'ensemble en une page

Comment lire ce document. Chaque section suit le même schéma : *quel problème, quelle réponse, comment ça marche* (schéma), *où dans le code*. Les chiffres de bascule proviennent des mesures exécutables (`scripts/*_measure.sh`) sur la stack de développement réelle (docker-compose : etcd + Patroni/Spilo).

1. Le problème que résout Mission C

La mission porte l'**infrastructure et la résilience** du cluster AMANE. Les autres missions construisent des fonctionnalités ; Mission C garantit qu'elles tiennent debout quand tout va mal : crash d'un nœud, coupure réseau entre sites, machine compromise, latence.

Quatre exigences structurent tout le code — et chacune a reçu une réponse mécanique précise plutôt qu'une promesse :

Exigence	Réponse apportée	Section
Il ne doit jamais y avoir deux écrivains simultanés (split-brain = corruption silencieuse)	Élection du leader par lease etcd + fencing applicatif : une écriture sur un nœud non-leader est refusée avant tout accès au journal	4-5
Les stocks doivent converger entre sites malgré un réseau lent ou coupé	delta-CRDT (totaux cumulés, fusion max) : ordre, doublons et trous deviennent mathématiquement inoffensifs	6
Une panne du primaire PostgreSQL doit être absorbée en moins de 5 s, sans perte	Superviseur maison : détection en 0,8 s, déverrouillage du lock Patroni, POST /failover — Patroni reste l'autorité	7
Rien ne doit être cassable en silence ni illisible en production	Contrat proto verrouillé par CI, tests à trois niveaux, chaos planifié, trace distribuée W3C, mTLS partout	3, 9-11

Le fil rouge : **chaque garantie est prouvée par un test ou une mesure ré-exécutable**, jamais affirmée. C'est pourquoi la dernière section de ce document parle autant de tests que les premières parlent d'algorithme.

2. Vue d'ensemble : ce que fait le système

L'unité de déploiement est le site (un atelier, une boutique en ligne...). Chaque site embarque la pile complète : l'orchestrateur Go expose l'API gRPC, etcd tient le consensus local, PostgreSQL (piloté par Patroni) porte les données avec un réplica synchrone, et un mesh WireGuard chiffre tout le trafic interne. Les sites se répliquent entre eux par un flux asynchrone de deltas — tolérant aux coupures.

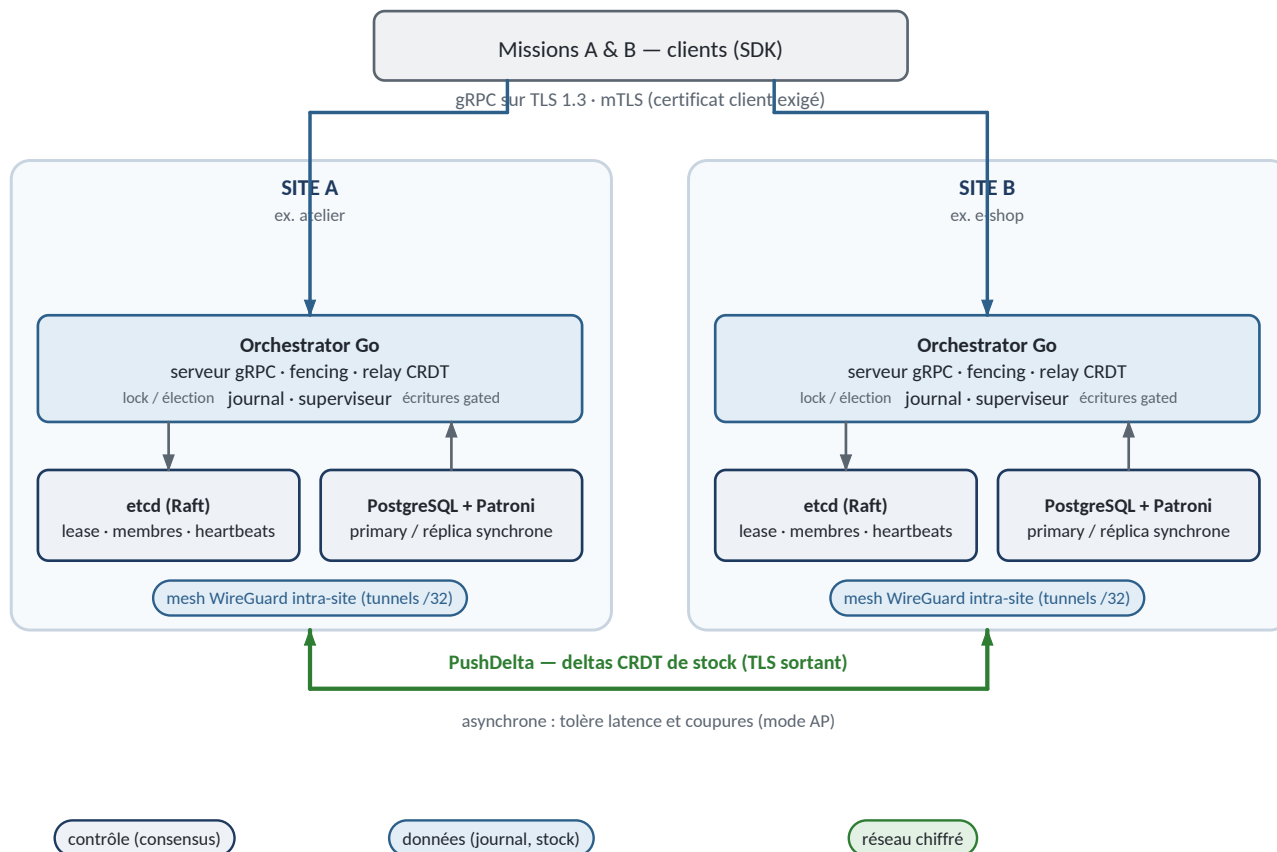


Figure 1 — Architecture globale : deux sites autonomes reliés par la réplication CRDT. Les flèches grises montrent qui contrôle quoi.

Les briques logicielles

Paquet Go	Rôle dans une phrase	Interface
consensus/	Élection du leader (lease 30 s), registre des machines, calcul du quorum	etcd v3
grpcserver/	Expose les 6 RPC du contrat ; applique le fencing et route vers le relay/journal	gRPC
replication/	PN-Counter CRDT (relay), journal append-only idempotent, propagateur périodique	interne
supervisor/	Détecte un crash de primary en < 1 s et déclenche le failover Patroni	REST + etcd
mesh/	Découverte des pairs via etcd et génération de wg0.conf (0600)	WireGuard
telemetry/	Propage l'en-tête W3C traceparent et corrélacionne les logs JSON	gRPC metadata
cmd/orchestrator	Câblage : TLS, interceptors, leadership, superviseur, propagateurs	main()
cmd/wgmesh	Outil runtime du mesh : publication, découverte, écriture de wg0.conf	CLI

Le plan mémoire du cluster : six familles de clés etcd

Tout l'état partagé vit sous des préfixes etcd explicites — c'est la mémoire commune du cluster. Retenir ces clés, c'est comprendre le système :

Clé etcd	Qui l'écrit	Signification
/amane/leader	consensus.Leadership	qui détient le droit d'écrire (fencing applicatif)
/amane/members/	Enroll / NotifyRevocation	registre des machines enrôlées → quorum
/service//leader	Patroni (DCS)	lock du primary PostgreSQL — autorité de promotion
/amane/supervisor/hb/	agent heartbeat local	« mon nœud est vivant et primary » (lease 2 s)
/amane/supervisor/force-right	superviseurs	lock « droit de forcer » : un seul acteur agit
/amane/mesh/nodes/	outil wgmesh	info publique WireGuard (jamais de clé privée)

Distinction cruciale : **/amane/leader** (fencing applicatif, géré par notre code) et **/service/<scope>/leader** (lock PostgreSQL, géré par Patroni) sont deux verrous différents. Le superviseur libère le second au moment du failover — jamais l'inverse (section 7).

3. Le contrat gRPC : la frontière versionnée

Mission C est un serveur ; les Missions A et B sont ses clients. Tout passe donc par un contrat protobuf partagé (`proto/amane/framework/v1/framework.proto`), généré avec `buf` et committé. La règle est absolue : **ne jamais renuméroter un champ ni changer son type** — cela casserait en silence les SDK déjà déployés. La CI rejette toute rupture (`buf breaking`).

Les six opérations du service

RPC	Rôle	Sens
Ping	// Ping : liveness du service (jalon 1)	interne/transverse
Enroll	Enroll : enrôlement d'une machine (Interface 1 A ↔ C, membership)	A ↔ C
Write	Write : chemin d'écriture (Interface 3 B ↔ C)	B ↔ C
Read	Read : lecture du journal (Interface 3 B ↔ C)	B ↔ C
NotifyRevocation	// Déclenche le recalcul du quorum côté C	A ↔ C
PushDelta	// max-par-nœud converge quel que soit l'ordre ou les doublons	interne/transverse

Deux conventions traversent ce contrat : les erreurs sont toujours des `status.Error` (codes .X) gRPC propres (jamais d'erreur brute), et le champ `encrypted_payload` n'est jamais autre chose que du chiffré produit par la Mission A — le serveur C ne voit jamais de clair et ne logue que des longueurs.

Ajouter un RPC est sûr quand il est **additif** : c'est comme cela qu'est né `PushDelta` (section 6) — nouveaux messages, nouveaux numéros de champs, anciens intacts, code généré committé.

4. Un seul écrivain : consensus et fencing

Le problème. Si deux nœuds se croient simultanément maîtres des écritures (après une micro-coupure réseau, par exemple), chacun numérottera les entrées à sa façon : corruption silencieuse du journal. Il faut un mécanisme où perdre le droit d'écrire est *instantané et irréfutable*.

La réponse : une lease etcd. Le droit d'écrire est matérialisé par une session dont la vie dépend d'un keepalive continu. Tant que le nœud bat le cœur, il est leader ; dès que la lease expire — crash, coupure, arrêt — la clé disparaît d'elle-même et le nœud sait localement (`IsLeader()` passe à `false`) qu'il doit refuser d'écrire. Aucune négociation, aucune fenêtre ambiguë.

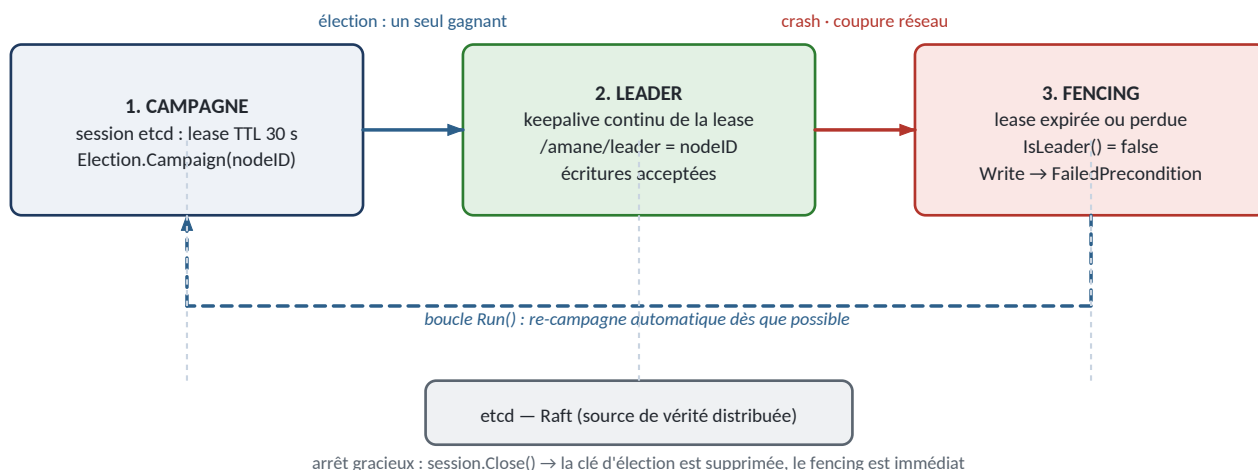


Figure 2 — Cycle de vie du leadership : campagne → leader → fencing → re-campagne. La boucle tourne tant que le processus vit.

Pourquoi c'est sûr. L'élection repose sur l'`Election` d'`etcd` (`concurrency.Election`) : exactement un gagnant par instance de lease. Un ancien leader qui revient après une partition ne peut pas récupérer le trône tant qu'une autre session le détient ; et pendant qu'il était isolé, son `IsLeader()` était déjà `false` — il n'a rien écrit. C'est le fencing « lease-based » classique, appliqué au chemin d'écriture gRPC.

- **Côté code** : `consensus/lease.go` (`Run/IsLeader`), clé `/amane/leader`, TTL 30 s.
- **Côté serveur** : `grpcserver/server.go` — `Write` consulte `WithLeadership` avant le journal.
- **Côté preuve** : `TestLeadershipFencingAgainstEtcd` (etcd réel) : l'ancien leader ne peut plus écrire après réélection.

5. Le chemin d'écriture : gated et idempotent

Le RPC Write est le point unique où les données entrent. Deux protections s'y cumulent : le **fencing** (section 4) qui filtre les non-leaders, et l'**idempotence** du journal qui rend les rejeux inoffensifs.

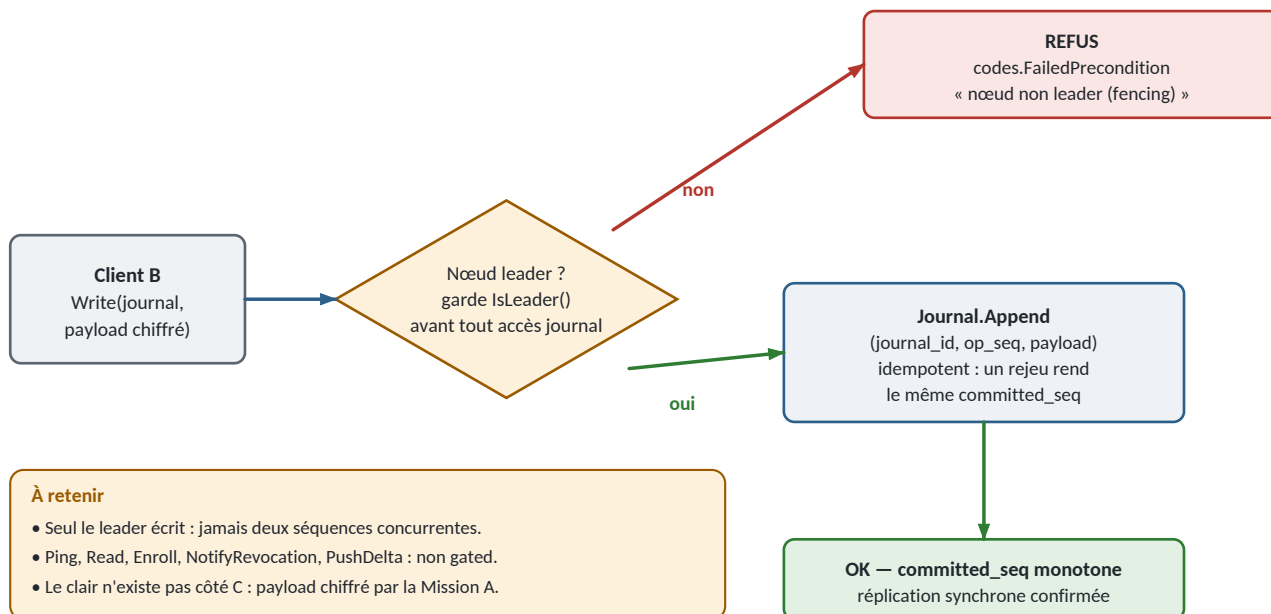


Figure 3 — Chemin d'écriture : la garde IsLeader() passe avant tout accès au journal ; un rejeu (même journal/site/op_seq) rend le seq déjà committé.

Le journal en deux idées. Un map `journal_id` → entrées append-only, et une clé de déduplication (`journal_id`, `site_id`, `op_seq`) : si le client renvoie la même opération (timeout, retry réseau), le serveur répond le seq déjà committé au lieu de dupliquer. Les seq attribués sont monotones par journal — le contrat promis aux lecteurs.

Et la lecture ? Read n'est pas gated : lire des données un peu anciennes sur un réplica vaut mieux que ne plus pouvoir lire du tout (cohérence ultime assumée pour les lectures). Seule l'écriture modifie l'histoire, donc seule l'écriture exige l'unicité du leader.

Ce qui circule — et ce qui ne circule jamais

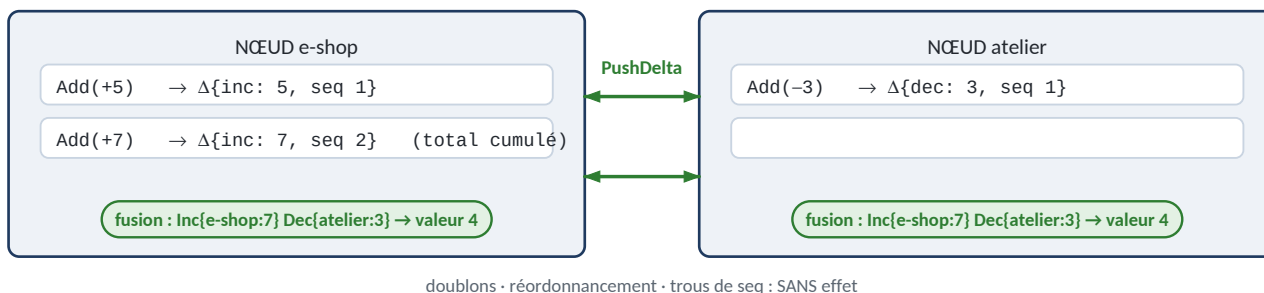
Donnée	Traitement côté C
payload chiffré	stocké tel quel, logué en longueur uniquement (jamais en clair)
op_seq client	sert à la déduplication ; le seq committé est attribué par le serveur
clé publique AK (Enroll)	empreinte SHA-256 (8 octets) stockée ; la clé elle-même n'est pas persistée
identifiant de clé révoquée	seul le fingerprint circule dans NotifyRevocation

6. Réplication multi-site : le delta-CRDT (mode AP)

Le problème. Les quantités de stock doivent être identiques sur tous les sites, mais les liens inter-sites sont lents et parfois coupés. Exiger une écriture synchrone entre sites serait une sentence : chaque coupure figerait tout. On choisit donc la disponibilité (mode AP) — à condition que la convergence soit *mathématiquement* garantie.

L'idée géniale mais simple des CRDT. Un delta n'emporte pas « +2 pièces » (un incrément relatif, qui casse si répété ou réordonné) mais le **total cumulé** de l'émetteur depuis son origine : « mes entrées totales = 7 ». Fusionner, c'est prendre le max par nœud émetteur. Max est commutatif, associatif et idempotent : peu importe l'ordre d'arrivée, les doublons ou les trous — tous les nœuds finissent avec exactement le même état. Ce n'est pas une espérance, c'est un théorème.

Scénario : stock partagé. e-shop : +5 puis +7 — atelier : -3. Attendu partout : 4.



Un delta porte le TOTAL cumulé de l'émetteur (pas un incrément relatif). La fusion « max par nœud émetteur » est alors
commutative · associative · idempotente convergence garantie (propriété AP des CRDT state-based)

Figure 4 — Exemple de convergence : +5 puis +7 d'un côté, -3 de l'autre ; chaque nœud fusionne par max et obtient 4, quel que soit le trafic réseau.

La livraison : un propagateur, pas une prière. Chaque nœud garde ses deltas non accusés dans un *pending* et un Propagator les pousse toutes les secondes vers chaque pair configuré. L'accusé porte la dernière seq appliquée par le pair : il purge le pending (Confirm). Une erreur réseau n'efface rien — le tick suivant retente. Au pire, on retarde ; jamais on ne perd.

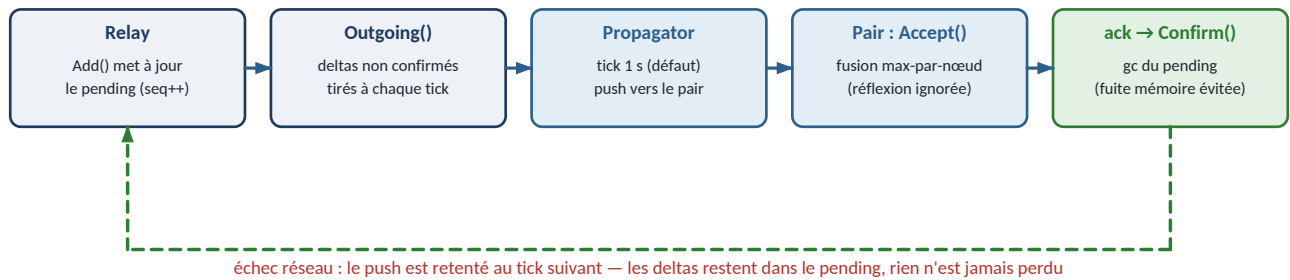


Figure 5 — Boucle de livraison d'un delta, de Add() jusqu'au gc du pending.

Situation réseau	Effet sur le système
delta reçu deux fois	aucun : $\max(\{7\}, \{7\}) = 7$ (idempotence)
deltas reçus en désordre	aucun : le max ne dépend pas de l'ordre (commutativité)
seq manquants (trous)	aucun : la seq ne sert qu'à l'accusé côté émetteur
pair injoignable des heures	retard seulement : le pending grossit puis se vide à la reconnexion
nos propres deltas revenus (réflexion mesh)	ignorés explicitement dans Accept()

Frontières honnêtes. Ce compteur PN-CRDT sert aux quantités de stock — un domaine où la convergence compte plus que l'instantanéité. Le journal d'écriture, lui, reste centralisé par leader (section 5) : deux régimes de cohérence choisis sciemment selon la donnée. Preuve live du repo : e-shop +5 puis +7 → ack seq 2, valeur 7 ; atelier -3 → valeur 4 partout.

7. Haute disponibilité < 5 s : le superviseur (option C)

Le problème. PostgreSQL bascule grâce à Patroni, mais Patroni s'appuie sur une lease DCS dont le plancher est $\text{ttl} \geq 20 \text{ s}$ (validé dans `patroni/config.py`, quel que soit le DCS). En cas de crash brutal, la promotion naturelle attend donc $\sim 21 \text{ s}$ — quatre fois trop pour le SLO. Abaisser ce TTL affaiblirait la protection anti-split-brain : rejeté.

La réponse retenue : détecter vite sans décider à la place de l'expert. Un petit superviseur en Go, déployé sur chaque nœud, sonde toutes les 500 ms l'API REST Patroni et publie un **heartbeat propre** dans etcd avec une lease de 2 s — indépendante du plancher de 20 s. Quand crash il y a (REST muet ET heartbeat disparu), il déverrouille la situation en deux gestes : supprimer le lock Patroni du défunt, puis demander la promotion. **Patroni reste seul juge de qui promouvoir** (réplica synchrone → zéro perte) : le superviseur n'est pas une couche HA parallèle, juste un déclencheur rapide.

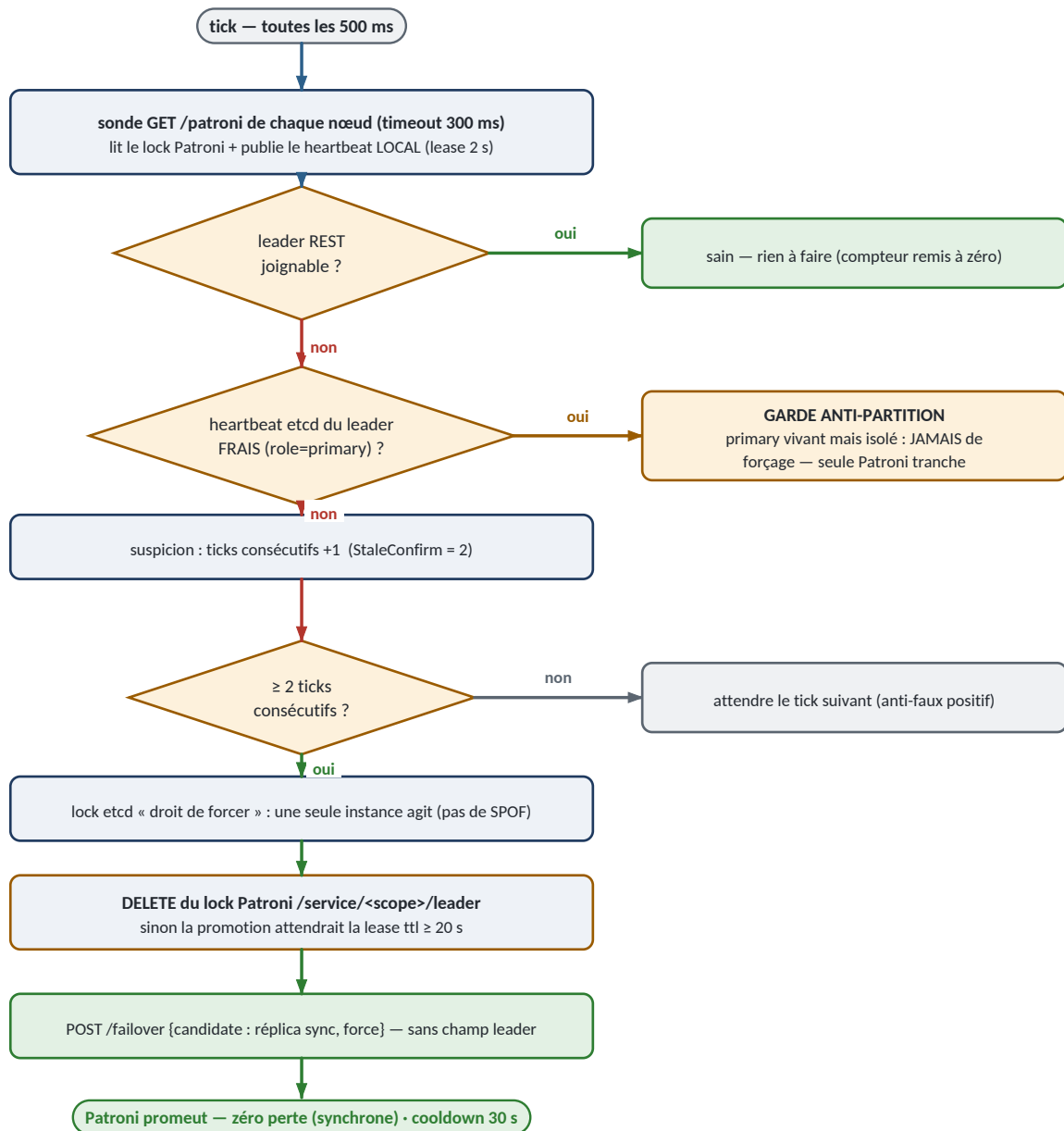


Figure 6 — L'arbre de décision à chaque tick. La garde anti-partition (ambre) distingue un nœud mort d'un nœud vivant mais isolé.

La garde anti-partition : le cœur subtil du dispositif

Couper le REST d'un primary ne le tue pas. Si le superviseur forçait sur la seule foi du REST muet, un simple incident réseau créerait deux primaires. D'où la règle : **forcer exige DEUX preuves de mort** — REST muet ET heartbeat etcd absent/périmé. Un primary isolé continue de battre son cœur : la garde bloque, Patroni tranche.

Subtilité de déploiement : chaque superviseur ne publie/ne supprime le heartbeat que de **son propre nœud** (LocalNode). Sinon, quand SON lien REST tombe, il effacerait le heartbeat du primary et neutraliserait la garde. Ce piège a été attrapé par un test d'intégration avant d'atteindre la production.

Pourquoi supprimer le lock change tout

Un POST /failover « force » seul suffit rarement : tant que le lock Patroni existe, la promotion reste bornée par l'expiration de la lease (≥ 20 s). Supprimer explicitement la clé /service/<scope>/leader — safe car le heartbeat stale prouve que le détenteur ne se battra pas — libère la promotion immédiatement. Le candidat choisi est un réplica joignable, prioritairement sync_state=sync (zéro perte).

Les mesures qui closent le débat

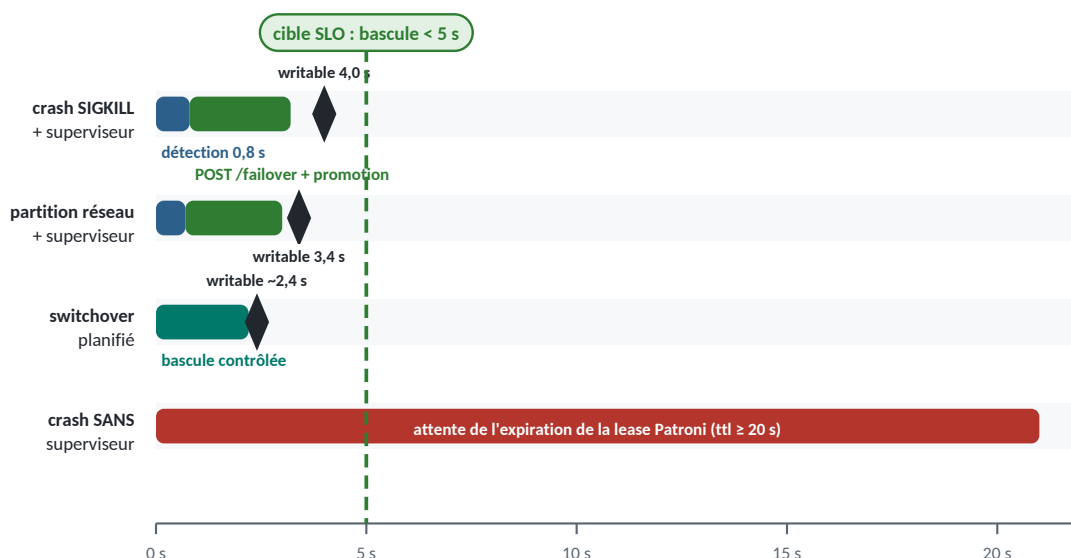


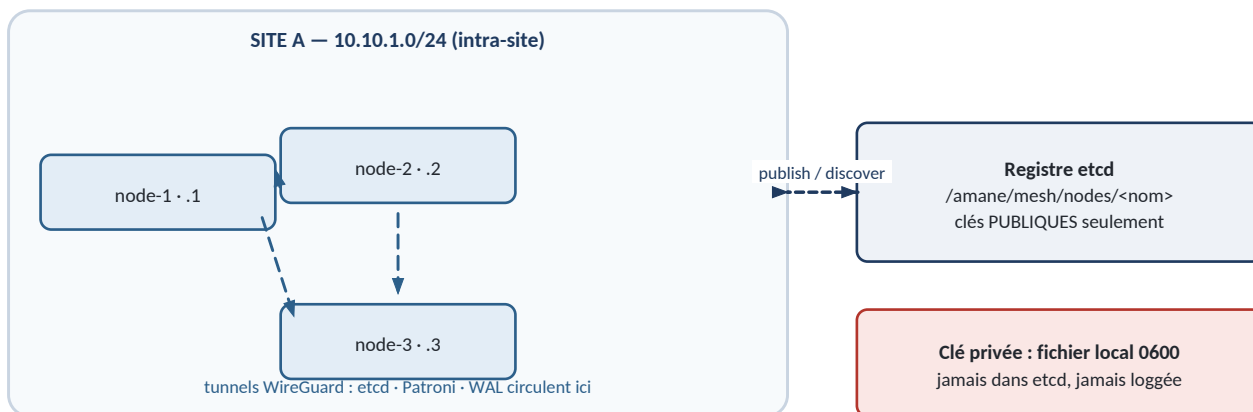
Figure 7 — Temps de bascule mesurés sur la stack réelle (scripts/failover_measure.sh, switchover_measure.sh). Fencing et zéro perte vérifiés à chaque exécution.

Scénario	Détection	Failover	Writable	Cible < 5 s
Switchover planifié	—	~2,2 s	~2,4 s	tenue
Crash SIGKILL + superviseur	0,8 s	3,2 s	4,0 s	tenue
Partition totale + superviseur	0,7 s	3,0 s	3,4 s	tenue
Crash sans superviseur	—	~21 s	~22 s	hors SLO (plancher Patroni)

Invariant vérifié à chaque mesure : synchronous_commit=on, standby synchrone, l'ancien primary ne recapture jamais (fencing), zéro transaction acquittée perdue.

8. Mesh WireGuard intra-site

Le trafic sensible d'un site — réplication WAL, heartbeats etcd, API Patroni — circule sur un VPN WireGuard interne. L'inter-site, lui, reste sur du TLS sortant : le mesh n'est volontairement **pas** la colonne vertébrale mondiale, juste le bouclier local.



AllowedIPs /32 strict (0.0.0.0/0 refusé au rendu) · PersistentKeepalive 25 s (NAT) · IP virtuelle stable indépendante du rôle → aucun rechiffrement réseau au failover

Figure 8 — Le runtime wgmesh publie l'information publique dans etcd, découvre les pairs et régénère wg0.conf uniquement si le contenu change.

Trois choix de sécurité dignes d'attention. D'abord, **AllowedIPs /32** : chaque pair n'autorise que l'IP virtuelle précise de l'autre — une route globale 0.0.0.0/0 serait rejetée au rendu (ErrDangerousAllowedIPs), impossible de détourner le trafic par une config bâclée. Ensuite, l'**IP virtuelle stable par nœud** (10.10.<site>.<index>) : un failover change un rôle, jamais une adresse — rien à reconfigurer au moment critique. Enfin, la **clé privée reste locale** : fichier 0600, jamais dans etcd, jamais dans les logs ; seul le matériel public circule.

9. Sécurité transverse : TLS 1.3, mTLS, secrets

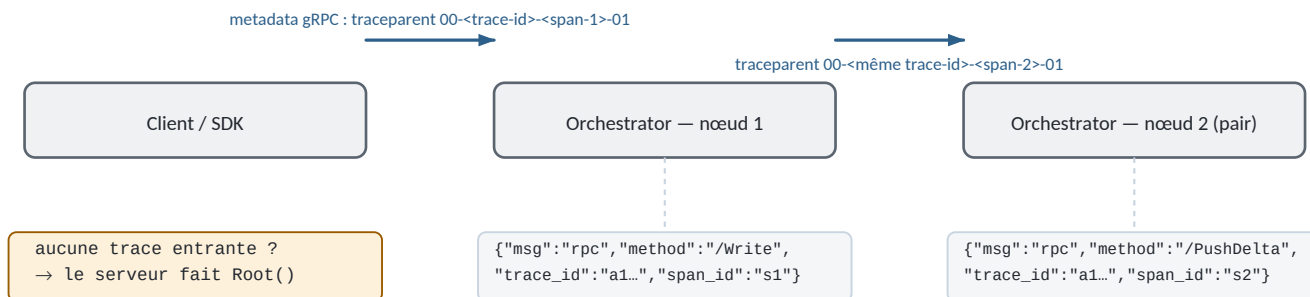
La posture tient en une phrase : **chiffrer par défaut, ne jamais exposer un secret, minimiser ce qui est cru.**

Surface	Règle appliquée	Preuve
Transport gRPC	TLS 1.3 minimum (MinVersion=1.3) — TLS 1.2 rejeté	TestCredentials ; openssl s_client -tls1_2 échoue
Authentification	mTLS : certificat client exigé et vérifié contre la CA interne	ClientAuth=RequireAndVerifyClientCert
Payloads	chiffrés par la Mission A ; C ne voit/logue que du chiffré	revues de code + tests
Clés AK (enrôlement)	seules clés publiques circulent ; empreinte SHA-256 comme ID	consensus.Fingerprint
PKI tls/	réservée au développement (clés 0600, jamais committée en prod)	règle AGENTS.md n° 6
Production (à venir)	AC racine hors ligne, ACs intermédiaires par site, OCSP/CRL, DNSSEC sortant	planifié hors POC

Le principe directeur : une règle de sécurité qui n'est pas vérifiée par un test n'existe pas. Chaque ligne ci-dessus renvoie vers un test ou une commande de contre-vérification exécutable.

10. Observabilité : la trace distribuée W3C

Dans un système à trois sites, « les logs » d'un incident sont éparpillés sur plusieurs machines. La réponse standard est OpenTelemetry ; la réponse Mission C est une version **forward-path** sans dépendance : le format W3C traceparent, transporté dans les métadonnées gRPC, avec génération d'une racine si l'appelant n'en porte pas. Le jour où un vrai SDK OTLP est branché, aucun point d'ancrage ne change.



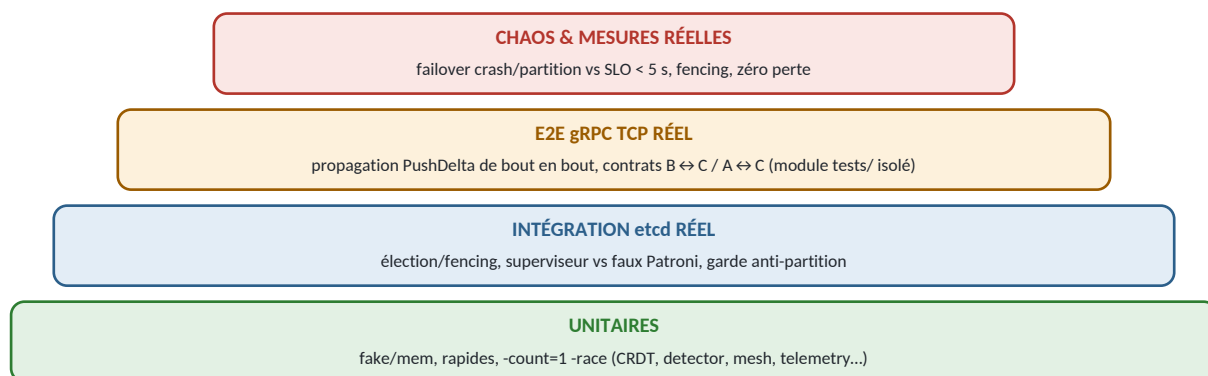
Le même trace-id relie les logs des deux nœuds : un Write et sa réplication deviennent traçables de bout en bout.
 Zéro dépendance OTel : format W3C standard — un SDK OTLP se branche plus tard sans changer les points d'ancrage.
 Chaque span ajoute aussi duration_ms aux logs JSON (slog) : corrélation du timing failover / lease entre nœuds.

Figure 9 — Un trace-id unique traverse les nœuds ; chaque saut crée un span enfant et enrichit les logs JSON (slog) de durée et d'identifiants.

Trois intercepteurs habillent chaque appel serveur, dans l'ordre : **télémetrie** (extrait/injecte le traceparent), **logging** (méthode, code, latence, trace_id/span_id), **recovery** (transforme un panic en codes.Internal — le serveur ne meurt jamais pour un handler fautif). Côté client, l'intercepteur réinjecte l'en-tête vers le pair : c'est lui qui rend le chemin PushDelta traçable de bout en bout.

11. La preuve par les tests et la CI

La philosophie : **chaque garantie précédente a un test qui la trahit si on la casse**. Les tests sont organisés en pyramide — vite et souvent en bas, réellement et rarement en haut — et la CI refuse de fusionner si un étage faiblit.



Portes CI : build + vet · couverture agrégée ≥ 70 % (plancher 60 % par paquet) · buf lint/breaking · testcontainers · govulncheck + gosec · benchmarks vs baseline

Figure 10 — Pyramide de tests et portes CI associées.

Les tests d'intégration utilisent un **etcd réel** (variable AMANE_TEST_ETCD ; testcontainers en CI) — mocker etcd testerait nos hypothèses sur etcd, pas etcd. Les tests cross-mission vivent dans un **module Go séparé** (tests/) qui consomme le produit : ils verrouillent les contrats B ↔ C et A ↔ C tels qu'un client les voit, pas tels que le code interne les arrange.

Workflow CI	Déclencheur	Ce qu'il bloque
proto-contract	push/PR	buf lint + breaking : toute rupture du contrat
build-go	push/PR	build + vet + tests + couverture $\geq 70\%$ (plancher 60 %/paquet)
integration-tests	push/PR	contrats B $\leftrightarrow$ C / A $\leftrightarrow$ C / write-gated sur etcd réel (testcontainers)
chaos-tests	PR consensus-replication-supervisor	failover crash, garde anti-partition, fencing, convergence CRDT
benchmark	PR chauds + cron	micro-bench vs baseline.json ; failover macro vs SLO < 5 s
security-scan	push/PR + cron	govulncheck + gosec (high bloquant)

Le plancher de couverture **par paquet** (60 %) complète le seuil agrégé (70 %) : un paquet à 40 % ne peut plus être masqué par son voisin à 90 %. Mesures actuelles : replication 95,7 %, telemetry 93,2 %, grpcserver 85,3 %, mesh 83,2 %, supervisor 80,2 %, consensus 73,8 % — agrégat ~72 %. Objectif annoncé : 75 % puis 80 %.

12. Synthèse : six principes directeurs

Si l'on ne devait retenir que la logique — celle qui explique chaque ligne de code — ce seraient ces six principes :

1 • L'état critique a un propriétaire unique	Le droit d'écrire est une lease, pas une opinion. Qui perd sa lease perd son droit, instantanément et sans ambiguïté (sections 4-5).
2 • La convergence vaut mieux que la coordination	Entre sites, on ne verrouille pas : on fusionne. Des totaux cumulés + un max par émetteur rendent les incidents réseau arithmétiquement inoffensifs (section 6).
3 • Détecter vite, décider sobrement	Le superviseur apporte la vitesse (0,8 s) ; Patroni conserve l'autorité (promotion synchrone, zéro perte). Deux preuves de mort avant tout forçage (section 7).
4 • Les contrats sont sacrés	Le proto est la loi commune : additif seulement, breaking interdit, vérifié en CI. Les erreurs sont typées gRPC, jamais ambiguës (section 3).
5 • Pas de secret exposé, jamais	Clés privées locales 0600, payloads chiffrés de bout en bout, fingerprints au lieu de clés, mTLS obligatoire (sections 8-9).
6 • Une affirmation sans test n'est pas une fonctionnalité	Unitaires rapides, intégrations sur etcd réel, chaos planifié, benchmarks contre baseline, seuils bloquants — la confiance se construit, elle ne se déclare pas (section 11).

Et la promesse finale du système, celle que les mesures valident à chaque exécution : **un cluster qui écrit à un seul endroit, se réplique partout, survit à la mort de son primaire en moins de cinq secondes sans perdre une transaction, et raconte tout ce qu'il fait dans des traces corrélées.**